

LOAD AI

Free, Free-Tier, and Open-Source Provider Strategy

*Image Understanding, Editing, Manipulation, Character Consistency, Provider Routing, and
Fallback Architecture*

Prepared for Load AI development notes
Compiled from the chat discussion and code-ready developer prompts.

Table of Contents

1. 1. Executive Summary
2. 2. Core Truth: Free Software vs Free Compute
3. 3. Load AI System Architecture
4. 4. Full Developer Prompt: Image Consistency and Manipulation
5. 5. Free, Free-Tier, and Open-Source Provider List
6. 6. Recommended Routing Order
7. 7. Open-Source Image Workflow Engines
8. 8. Open-Source Image Consistency Tools
9. 9. Image Manipulation, Selection, Inpainting, Cutout, and Restoration Tools
10. 10. Vision Models, Prompt Builders, Audio, Video, and Hosting Options
11. 11. Developer Rules and Capability Map
12. 12. Copy-Paste Build Notes
13. 13. Practical Load AI Modes
14. 14. Source and Verification Notes

1. Executive Summary

This document consolidates the Load AI chat discussion into a developer-ready build reference. It focuses only on free, free-tier, and open-source options, while clearly separating free software from the compute required to run it.

The central strategic point is simple: Load AI should not rely on one provider. It should use a capability-based routing system that sends the right task to the right tool. Text-only models should never receive image editing requests. Vision models should analyze images before editing. Image generators should be checked for actual image output. Local or self-hosted open-source engines should handle serious manipulation, consistency, inpainting, outpainting, and advanced workflows.

- Use Puter.js and Pollinations as the easiest early free provider stack.
- Use Hugging Face, Cloudflare Workers AI, and free-tier providers as optional routing layers.
- Use ComfyUI, InvokeAI, AUTOMATIC1111, Forge, IOPaint, rembg, SAM, ControlNet, IP-Adapter, InstantID, Real-ESRGAN, GFPGAN, and CodeFormer as the serious open-source toolkit.
- Treat character consistency as an architecture problem, not merely a prompt problem.
- Never claim an image edit succeeded unless the app receives an actual image file, blob, or URL.

2. Core Truth: Free Software vs Free Compute

Yes, ComfyUI and many of the other tools discussed are free and open-source. The precise answer is that the software is free, but running the software can require compute, usually a GPU.

If the user runs the tool on a computer they already own, the software can operate without a cloud bill. If Load AI runs those tools for many users through a server, GPU hosting will cost money. This distinction matters because it prevents false expectations during product planning.

Software cost: free
 Model/tool code: usually free or open-source
 Running locally on your own computer: free after you own the computer
 Running in the cloud: usually free only with limited free tiers
 Running commercially at scale: not free because GPU/server costs

Tool	Free/Open-Source?	Does it cost to run?	Best use
ComfyUI	Yes, open-source	Free locally, cloud GPU costs money	Advanced image workflows and consistency
AUTOMATIC1111	Yes, open-source	Free locally, cloud GPU costs money	Stable Diffusion generation and editing
InvokeAI	Yes, free/open-source	Free locally, cloud GPU costs money	Creator-friendly generation and editing
IOPaint / LaMa Cleaner	Yes, free/open-source	Free locally, cloud GPU optional	Object removal and inpainting
rembg	Yes, open-source	Free locally	Background removal
SAM / SAM 2	Open-source code/checkpoints available	Free locally, heavier compute may be needed	Smart object selection and masks
ControlNet	Open-source	Free locally, needs diffusion backend	Pose, layout, and depth control

IP-Adapter	Open-source	Free locally, needs diffusion backend	Reference image consistency
Real-ESRGAN	Open-source	Free locally	Upscaling
GFPGAN / CodeFormer	Open-source	Free locally	Face and portrait repair

3. Load AI System Architecture

Load AI should be built as a modular provider router with separated pipelines. The system should understand images, construct prompts, route to the correct image tool, verify outputs, and fall back gracefully when free providers fail.

Required pipelines:

- A. Vision / Image Analysis Pipeline
- B. Prompt Builder / Consistency Builder
- C. Image Editing / Manipulation Pipeline
- D. Image Generation Pipeline
- E. Output Verification Pipeline
- F. Fallback Pipeline

4. Full Developer Prompt: Image Consistency and Manipulation

Copy and paste this into the developer notes or builder prompt for Load AI.

Build a multi-model AI image system that supports:

1. image understanding
2. image editing
3. image generation
4. image consistency
5. image manipulation
6. multi-image continuity
7. fallback routing when free models fail

This system must not assume one free AI model can do everything.

GOAL

Allow users to:

- upload one or more images
- ask what the image shows
- ask for edits
- keep characters, faces, outfits, props, and style consistent
- manipulate specific parts of an image
- generate new images based on prior images
- receive an edited image, newly generated image, or fallback prompt if no image model is available

=====

SYSTEM ARCHITECTURE

=====

Use separate pipelines:

- A. Vision / Image Analysis Pipeline
- B. Prompt Builder / Consistency Builder
- C. Image Editing / Manipulation Pipeline
- D. Image Generation Pipeline

Load AI Documentation | Free, Free-Tier, and Open-Source Provider Strategy

E. Output Verification Pipeline

F. Fallback Pipeline

```
=====
A. VISION / IMAGE ANALYSIS PIPELINE
=====
```

Use a vision-capable model only.

Use this for:

- describe this image
- what do you see
- identify objects
- identify style
- compare images
- detect consistency
- detect problems
- read visible text
- generate edit instructions from uploaded image
- extract character identity traits

The vision model should return structured JSON like:

```
{
  "image_description": "",
  "subjects": [],
  "character_count": 0,
  "character_profiles": [
    {
      "name_or_label": "",
      "gender_presentation": "",
      "estimated_age_range": "",
      "skin_tone": "",
      "face_shape": "",
      "eye_shape": "",
      "hair_color": "",
      "hair_style": "",
      "body_type": "",
      "distinctive_features": [],
      "clothing": [],
      "pose": "",
      "expression": "",
      "accessories": []
    }
  ],
  "objects": [],
  "background": "",
  "style": "",
  "palette": [],
  "composition": "",
  "lighting": "",
  "problems_detected": [],
  "recommended_edit_prompt": "",
  "can_edit_directly": true
}
```

If there are multiple reference images, compare them and return:

```
{
  "shared_identity_features": [],
  "style_consistency_notes": [],
  "differences_detected": [],
}
```

```
"continuity_recommendations": []
}
```

=====

B. PROMPT BUILDER / CONSISTENCY BUILDER

=====

Create a consistency-aware prompt builder.

This builder must convert:

- user request
- uploaded image(s)
- extracted vision traits
- prior character/style memory

into a clean prompt for image editing or generation.

The prompt builder must preserve:

- same character identity
- same face
- same hairstyle
- same clothing if requested
- same art style
- same color palette if requested
- same prop/object identity
- same environment if requested

Prompt builder must support the following modes:

1. strict_identity_preservation
2. moderate_identity_preservation
3. loose_inspiration_mode

Strict identity preservation example:

"Preserve the exact same character identity from the reference image. Keep the face shape, skin tone, hair texture, hairstyle, eye shape, age range, body type, and outfit design consistent. Maintain the same visual style and overall character appearance. Only change the facial expression to fear."

Moderate identity preservation example:

"Keep the same general character identity and style, but allow natural variation in pose and framing."

Loose inspiration example:

"Use the uploaded image as inspiration only. Do not replicate exactly."

=====

C. IMAGE EDITING / MANIPULATION PIPELINE

=====

Use an image-capable model only.

This pipeline is for:

- remove object
- replace object
- swap background
- change expression
- change pose
- recolor clothing
- add accessories
- fix hand or face issues
- expand image

Load AI Documentation | Free, Free-Tier, and Open-Source Provider Strategy

- crop image
- clean up artifacts
- change lighting
- restyle image
- inpaint a selected area
- outpaint the canvas
- make image more cinematic
- preserve character while changing scene

Supported manipulation types:

1. inpainting

Edit only selected region while preserving the rest.

2. outpainting

Expand image beyond existing borders.

3. object removal

Remove unwanted elements.

4. object replacement

Replace an item with a different item.

5. background replacement

Keep subject, change environment.

6. expression manipulation

Keep identity, alter emotional expression.

7. pose manipulation

Keep identity, alter pose.

8. style transfer / style locking

Keep same subject, change or preserve art style.

9. color / lighting manipulation

Change mood, exposure, color tone, time of day.

10. image cleanup

Improve detail, reduce artifacts, sharpen or restore.

If the selected model does not support region editing:

- generate a precise edit prompt
- preserve as much identity as possible
- state that region-level editing is unavailable
- try next provider automatically

=====

D. IMAGE GENERATION PIPELINE

=====

Use this when the user wants:

- a new image from scratch
- a new scene with the same character
- multiple scenes with continuity
- a new pose or setting based on prior references

If user wants continuity across several images:

- generate a character profile first
- store that profile
- attach reference images to future prompts
- include locked identity traits in every generation prompt

For multi-image continuity, use:

- one or more reference images
- saved character profile
- saved style profile
- saved wardrobe profile
- saved prop/background notes

Each new scene prompt should explicitly state:

- keep same character identity
- keep same art style
- keep same clothing or change only what is specified
- keep same hairstyle unless otherwise requested
- preserve color and mood continuity if requested

=====

E. OUTPUT VERIFICATION PIPELINE

=====

After every image generation or edit, run a second-pass vision check.

Compare result against:

- original reference image
- saved character profile
- user instructions

Return structured verification:

```
{
  "identity_match_score": 0,
  "style_match_score": 0,
  "instruction_match_score": 0,
  "differences_found": [],
  "passed": true
}
```

If result fails consistency threshold:

- try one automatic retry with a stronger preservation prompt
- if it still fails, show user the result and offer:
 1. Retry with stricter consistency
 2. Use fallback model
 3. Copy optimized prompt

=====

F. FALLBACK PIPELINE

=====

If no free image editing model is available:

- do not pretend editing succeeded
- return a high-quality optimized prompt instead
- explain clearly that the current provider cannot return an image

Fallback format:

```
{
  "success": false,
  "type": "fallback_prompt",
  "message": "No image editing model is currently available. Use the prompt below in a compatible image editor.",
  "optimized_prompt": ""
}
```


=====

MODEL CAPABILITY MAP

=====

Add a capability registry like:

```
const MODEL_CAPABILITIES = {
  "modelA": {
    text: true,
    vision: false,
    imageGeneration: false,
    imageEditing: false,
    inpainting: false,
    outpainting: false,
    consistencySupport: false
  },
  "modelB": {
    text: true,
    vision: true,
    imageGeneration: true,
    imageEditing: true,
    inpainting: true,
    outpainting: false,
    consistencySupport: true
  }
};
```

Do not send image editing requests to text-only models.
Do not send vision tasks to text-only models.
Do not assume image generation means image editing is supported.
Do not assume image editing means region editing is supported.

=====

PROVIDER ROUTING

=====

Create provider routing with automatic fallback.

Preferred routing order:

1. Puter image-capable model
2. Puter vision-capable model
3. Gemini image model if exposed through Puter
4. Pollinations free image generation
5. Hugging Face image inference endpoint
6. Replicate community/free model
7. local browser-based fallback
8. optimized prompt fallback

Pseudo-code:

```
async function routeImageTask({
  taskType,
  prompt,
  referenceImages = [],
  editMask = null,
  consistencyMode = "strict"
}) {
  const providers = [
    "puter-image",
    "puter-vision",
    "gemini-image",
    "pollinations",
```

```
"huggingface",
"replicate",
"fallback-prompt"
];

for (const provider of providers) {
  try {
    const result = await runProvider(provider, {
      taskType,
      prompt,
      referenceImages,
      editMask,
      consistencyMode
    });

    if (result && result.success) {
      return result;
    }
  } catch (err) {
    console.warn(`Provider failed: ${provider}`, err);
  }
}

return {
  success: false,
  type: "fallback_prompt",
  message: "All image providers failed. Returning optimized prompt.",
  optimizedPrompt: prompt
};
}
```

=====

REFERENCE IMAGE MEMORY

=====

Add persistent reference storage for consistency.

Store:

- character profile
- style profile
- background profile
- wardrobe profile
- prop profile

Each saved project can include:

```
{
  "projectId": "",
  "characterProfiles": [],
  "styleProfile": {},
  "wardrobeProfiles": [],
  "propProfiles": [],
  "referenceImageUrls": []
}
```

When user says:

- "keep same character"
- "same woman as before"
- "same outfit"
- "same style"
- "make next scene match previous"

Load AI Documentation | Free, Free-Tier, and Open-Source Provider Strategy

the app must fetch previous references automatically.

=====

CONSISTENCY RULES

=====

When consistencyMode = strict:

- preserve face shape
- preserve eye shape
- preserve skin tone
- preserve hairstyle
- preserve distinctive features
- preserve clothing silhouette unless changed
- preserve visual style
- preserve age appearance
- preserve body type

When consistencyMode = moderate:

- preserve identity and overall style
- allow some framing and pose variation

When consistencyMode = loose:

- use reference as inspiration only

=====

MANIPULATION TOOLS IN UI

=====

Add clear user-facing tools for:

- Remove
- Replace
- Change background
- Expand image
- Crop
- Restyle
- Change expression
- Change pose
- Recolor
- Add object
- Remove object
- Repair face/hands
- Match previous character
- Match previous style

Optional advanced UI:

- brush mask tool
- lasso selection
- preserve face toggle
- preserve outfit toggle
- preserve style toggle
- strict consistency toggle

=====

TASK DETECTION

=====

If user uploads an image and asks:

- "what is this"
- "describe this"
- "is this the same character"

Route to vision pipeline.

```

If user says:
- "edit"
- "fix"
- "remove"
- "change"
- "replace"
- "make consistent"
- "keep same character"
- "same face"
- "new scene with same person"
Route to consistency-aware image editing or generation pipeline.

=====
FINAL REQUIREMENT
=====

This app must support:
- image understanding
- image editing
- image generation
- image consistency
- image manipulation
- multi-image continuity
- fallback prompt generation

It must intelligently separate vision, prompt building, editing, and consistency handling instead
of expecting one free model to do everything.

```

5. Free, Free-Tier, and Open-Source Provider List

The following list is limited to free, free-tier, or open-source options. Some are free software but need compute. Some are free-tier cloud options with usage limits.

Provider	Best for	Load AI use
Puter.js	Frontend AI access, text, image generation, OCR, Gemini/GPT/Claude access where supported, cloud storage, possible vision workflows.	First provider to try for frontend generation, prompt building, and basic routing.
Pollinations AI	Free image generation, text generation, audio/video experiments, fast fallback generation.	Second image-generation fallback and quick creative provider.
Hugging Face Inference Providers	Open-source model access, vision models, image models, audio models, testing many models.	Open-source model connector with free-tier limits.
Hugging Face Spaces	Free demo apps, Gradio APIs, open-source model demos.	Optional model connector and prototype provider.
Cloudflare Workers AI	Free-tier serverless AI, lightweight backend routing, hosted open-source models.	Backend router layer and protected API bridge.
Google Colab Free Tier	Testing ComfyUI, Stable Diffusion, open-source models, temporary GPU experiments.	Development and testing only.

Kaggle Notebooks Free Tier	Free GPU experiments, model testing, notebooks.	Development and testing only.
Vercel Free Tier	Web hosting and serverless API routes.	PWA hosting and lightweight backend functions.
Netlify Free Tier	Static hosting and serverless functions.	Frontend deployment and basic API functions.
Supabase Free Tier	Database, storage, auth, project memory.	Reference image metadata, character profiles, user project storage.
Firebase Free Tier	Auth, database, storage, app hosting.	Optional backend system.

6. Recommended Routing Order

Vision / Image Understanding

15. Puter vision-capable model
16. OpenRouter free vision model, if used
17. Hugging Face vision model
18. Qwen2.5-VL local/server
19. LLaVA local/server
20. Florence-2 for structured detection/captioning

Image Generation

21. Puter.js image generation
22. Pollinations AI
23. Cloudflare Workers AI
24. Hugging Face model or Space
25. ComfyUI local/server
26. AUTOMATIC1111 / Forge / InvokeAI local/server

Image Editing

27. Puter image-capable model, only if it returns actual images
28. Gemini/image model through available free route, only if supported
29. ComfyUI inpainting/image-to-image
30. AUTOMATIC1111 / Forge inpainting
31. InvokeAI canvas editing
32. LaMa Cleaner / IOPaint for object removal
33. Fallback optimized prompt

Object Selection and Masks

34. SAM 2
35. SAM original
36. Grounding DINO
37. Florence-2
38. YOLO fallback

Character Consistency

- 39. IP-Adapter
- 40. InstantID
- 41. ControlNet
- 42. LoRA
- 43. Seed locking
- 44. Saved reference image memory
- 45. Post-output vision verification

Background Removal

- 46. rembg
- 47. BiRefNet
- 48. U2-Net
- 49. SAM 2 mask plus transparent export

Upscaling and Restoration

- 50. Real-ESRGAN
- 51. SwinIR
- 52. GFPGAN
- 53. CodeFormer

Voice and Audio

- 54. Piper TTS
- 55. Coqui TTS
- 56. eSpeak NG
- 57. Whisper / Whisper.cpp
- 58. Vosk

7. Open-Source Image Workflow Engines

Tool	Best for	Load AI feature/use
ComfyUI	Advanced image workflows, Stable Diffusion, Flux workflows, inpainting, outpainting, ControlNet, IP-Adapter.	Advanced image engine and serious consistency pipeline.
AUTOMATIC1111 Stable Diffusion WebUI	Stable Diffusion generation, inpainting, image-to-image, extensions, ControlNet.	Optional local/server backend.
Stable Diffusion WebUI Forge	Optimized Stable Diffusion workflows, lower VRAM, inpainting, ControlNet workflows.	Advanced local creator workstation provider.
InvokeAI	User-friendly generation, canvas editing, inpainting, outpainting.	Creator-friendly local/server image editor.

8. Open-Source Image Consistency Tools

Tool	Best for	Load AI feature/use
IP-Adapter	Reference image consistency, style consistency, character consistency, wardrobe/style reference.	Reference Lock, Style Lock, Character Reference Mode.
InstantID	Face identity consistency, same person across generated images.	Face Lock and Character Identity Lock.
ControlNet	Pose, depth, edge, layout, body position, composition preservation.	Pose and Layout Lock.
LoRA	Consistent character/style adapters, repeating a fictional character design, brand styles.	Character Pack, Style Pack, Wardrobe Pack.
DreamBooth	Training a model on a specific subject or style.	Advanced character training option.

9. Image Manipulation, Selection, Inpainting, Cutout, and Restoration Tools

Tool	Best for	Load AI feature/use
SAM 2	Object selection, person selection, mask creation, video object tracking.	Smart Select and Mask Builder.
SAM original	Image segmentation, mask creation, object selection.	Fallback segmentation tool.
Florence-2	Captioning, object detection, visual grounding, structured image understanding.	Image Inspector and Visual Metadata Extractor.
Grounding DINO	Text-guided object detection, selecting objects by phrase.	Text-to-Object Selector.
YOLO open-source models	Fast object and people detection.	Quick detection fallback.
IOPaint / LaMa Cleaner	Object removal, defect removal, inpainting, repair damaged areas.	Clean Remove, Object Eraser, Image Repair.
Stable Diffusion Inpainting	Replacing objects, changing clothing/backgrounds, filling masked areas.	Replace Object, Change Clothing, Replace Background.
SDXL Inpainting	Higher-quality inpainting and larger edits.	Advanced inpainting backend.
Flux Fill / Flux inpainting workflows	Modern inpainting workflows where available.	Future advanced edit route.
rembg	Background removal, transparent PNG, subject cutouts.	Background Cutout and Transparent PNG Maker.
U2-Net	Background removal and salient	Background removal model option.

	object detection.	
BiRefNet	High-quality subject segmentation and fine edge cutouts.	Premium-quality open-source cutout option.
Real-ESRGAN	Image upscaling and low-resolution enhancement.	Upscale and Enhance.
SwinIR	Restoration, denoising, super-resolution.	Restore Image.
GFPGAN	Face restoration and blurry portrait repair.	Face Repair.
CodeFormer	Face restoration and distorted AI face repair.	Portrait Repair.

10. Vision Models, Prompt Builders, Audio, Video, and Hosting Options

Tool	Best for	Load AI feature/use
Qwen2.5-VL	Image understanding, visual Q&A, OCR-style reading, multi-image analysis.	Vision Brain, Image Analyst, Consistency Checker.
LLaVA	Image chat, visual question answering, basic image description.	Fallback vision model.
MiniCPM-V	Smaller multimodal vision-language use.	Lightweight vision fallback.
BLIP / BLIP-2	Image captioning and image-to-text.	Caption generator and metadata generator.
OpenCLIP / CLIP	Image similarity, image search, style matching, reference comparison.	Similarity checker and consistency scoring helper.
Llama open models	Prompt rewriting, task routing, structured JSON, local reasoning.	Prompt builder and routing assistant.
Mistral open models	Prompt rewriting, structured outputs, fast text tasks.	Prompt optimizer and offline helper.
Qwen text models	Prompt generation, structured output, coding help.	Text reasoning provider.
Phi open models	Lightweight local reasoning and classification.	Lightweight fallback brain.
Piper TTS	Offline text-to-speech and narration.	Offline reader voice.
Coqui TTS	Open-source TTS experiments and character voices.	Experimental voice engine.
eSpeak NG	Lightweight offline speech.	Emergency offline speech fallback.
Whisper / Whisper.cpp	Speech-to-text, transcription, captions.	Voice input and subtitle generator.
Vosk	Offline speech recognition.	Offline voice command fallback.

AnimateDiff	Image-to-video style animation and short motion clips.	Experimental image animation backend.
Deforum Stable Diffusion	Prompt-based animation, camera movement.	Experimental animation generator.
Stable Video Diffusion	Image-to-video workflows.	Image animation provider where available.
Wan open video models	Open video generation workflows.	Future video generation backend.
FFmpeg	Video processing, frame extraction, compression, format conversion.	Media processing and export engine.

11. Developer Rules and Capability Map

```
const MODEL_CAPABILITIES = {
  "puter-image-model": {
    text: true,
    vision: true,
    imageGeneration: true,
    imageEditing: true,
    inpainting: false,
    outpainting: false,
    backgroundRemoval: false,
    segmentation: false,
    consistencySupport: "moderate",
    returnsImageFile: true,
    returnsImageUrl: true,
    returnsTextOnly: false
  },
  "pollinations": {
    text: true,
    vision: false,
    imageGeneration: true,
    imageEditing: false,
    inpainting: false,
    outpainting: false,
    backgroundRemoval: false,
    segmentation: false,
    consistencySupport: "loose",
    returnsImageFile: false,
    returnsImageUrl: true,
    returnsTextOnly: false
  },
  "text-only-model": {
    text: true,
    vision: false,
    imageGeneration: false,
    imageEditing: false,
    inpainting: false,
    outpainting: false,
    backgroundRemoval: false,
    segmentation: false,
    consistencySupport: false,
    returnsImageFile: false,
    returnsImageUrl: false,
    returnsTextOnly: true
  }
};
```

Rules:

1. Never send image-editing requests to text-only models.
2. Never claim an image was edited unless an actual image file, image blob, or image URL is returned.
3. If a provider returns text only, mark that provider as failed for image editing and try the next provider.
4. Keep a provider capability map.
5. Store reference images and character profiles.
6. Add Strict, Moderate, and Loose consistency modes.
7. Add automatic verification after every generated or edited image.
8. Add a fallback prompt if no free or open-source provider returns an image.

12. Copy-Paste Build Notes

Use free/open-source tools where possible, but separate software cost from compute cost.

ComfyUI, InvokeAI, AUTOMATIC1111, IOPaint, rembg, SAM, ControlNet, IP-Adapter, Real-ESRGAN, GFPGAN, and CodeFormer are free/open-source tools.

They can run for free on a local machine if the user owns the hardware.

If we run them for users through our own server, we will need GPU hosting, which is not free at scale.

For Load AI, build three modes:

1. Browser Mode: lightweight edits only
2. Free Provider Mode: Puter, Pollinations, Hugging Face, Cloudflare free-tier
3. Local Engine Mode: user runs ComfyUI/IOPaint/rembg/SAM locally for serious editing

13. Practical Load AI Modes

Browser Mode

Use for lightweight edits inside the PWA: image compression, resizing, cropping, metadata, prompt building, and possibly simple client-side utilities. This is best for iPad and mobile-friendly workflows.

Free Provider Mode

Use Puter.js, Pollinations, Hugging Face free-tier, Cloudflare Workers AI, and other limited free-tier routes. This is useful for early Load AI testing and lower-cost creative experimentation.

Local Engine Mode

For serious editing and consistency, the best path is a companion local engine running on a desktop, laptop, local server, or hosted machine. The PWA sends requests to this engine, and the engine runs ComfyUI, IOPaint, rembg, SAM, ControlNet, IP-Adapter, InstantID, and similar tools.

User installs Load AI Local Engine on a laptop/desktop.

Local Engine runs ComfyUI, IOPaint, rembg, SAM, ControlNet, IP-Adapter.

The Load PWA sends requests to the local engine.

No cloud GPU bill if the user owns the hardware.

14. Source and Verification Notes

The tool names and categories in this document should be verified again by the developer before production, because free-tier access, model availability, and licenses can change. The safest production design is therefore capability-based routing, not hard-coded assumptions.

- Puter.js documentation describes frontend AI access and image generation without traditional backend/API key setup for supported use cases.
- Pollinations describes free/community APIs for text, image, audio, and video generation.
- Hugging Face Inference Providers and Spaces provide access to open-source model experiments with free-tier limits.
- Cloudflare Workers AI has free-tier allocation and can work as a serverless routing layer.
- ComfyUI, AUTOMATIC1111, InvokeAI, IOPaint, rembg, SAM, ControlNet, IP-Adapter, Real-ESRGAN, GFPGAN, and CodeFormer are best treated as free/open-source software that still needs compute.